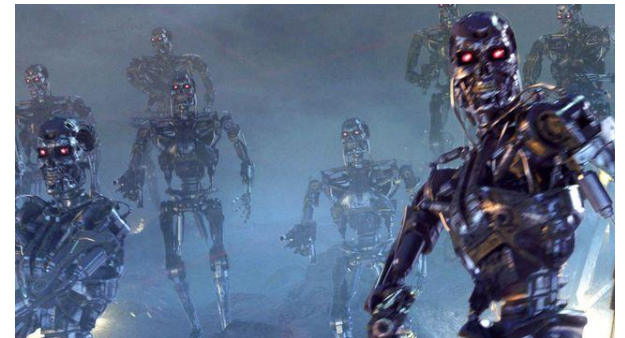
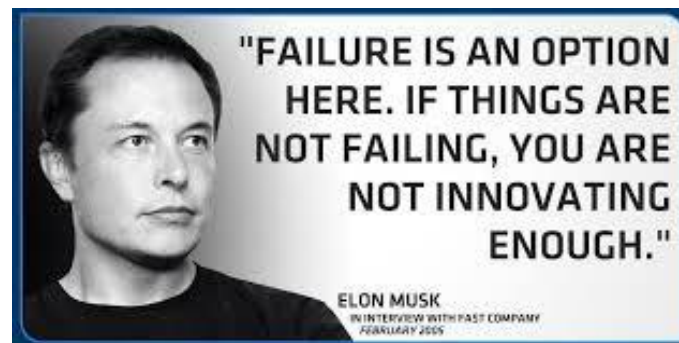




AGI et Finance : l'opportunité des réseaux neuronaux avancés(annexes)

28/03/2017 //////////////

Olivier Croissant



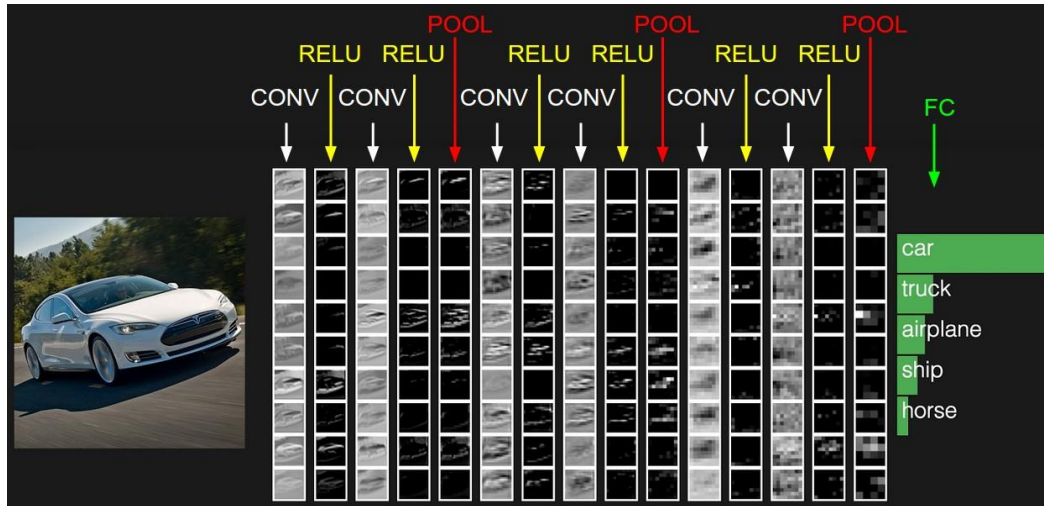
Let's try it !





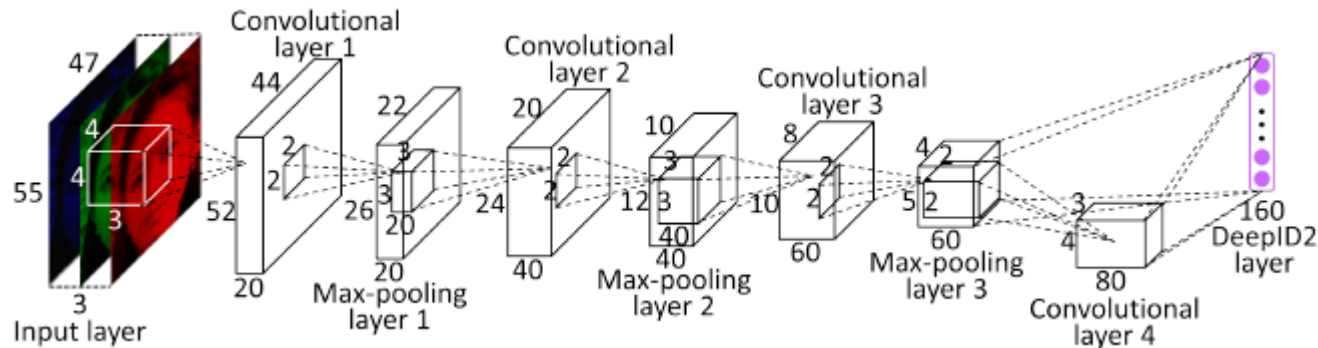
- 1. Fonctionnement de la vision humaine**
- 2. Réseaux convolutionnels**
- 3. Performance des réseaux convolutionnels en vision**
- 4. Chatbots**
- 5. Reinforcement learning**

Réseaux convolutionnels



Deux types de cellule:

1. Convolution
2. Pooling

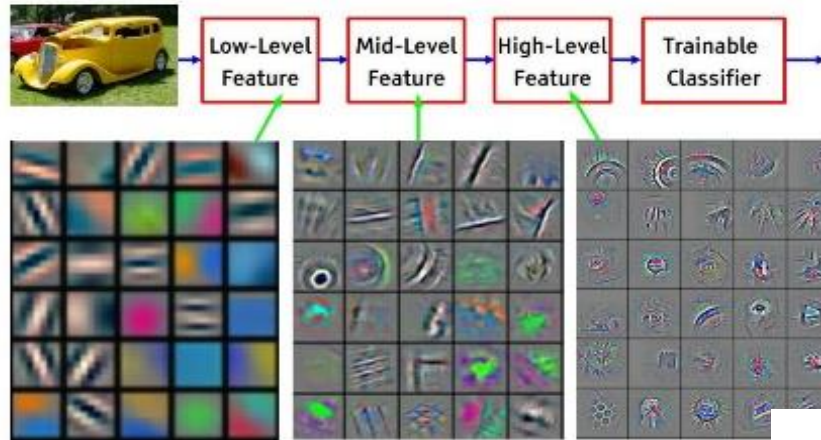


Rerseau convolutionnel : traditionnels

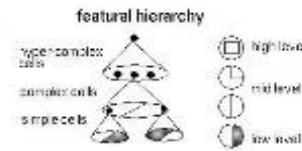


ConvNet : Interpretation

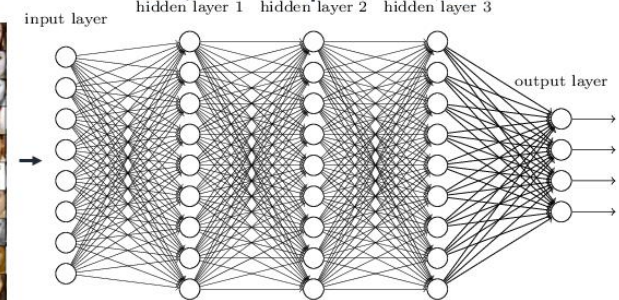
Chaque couche analyse un niveau d'agrégation



Feature visualization of convolutional net trained on ImageNet from [Zeller & Fergu



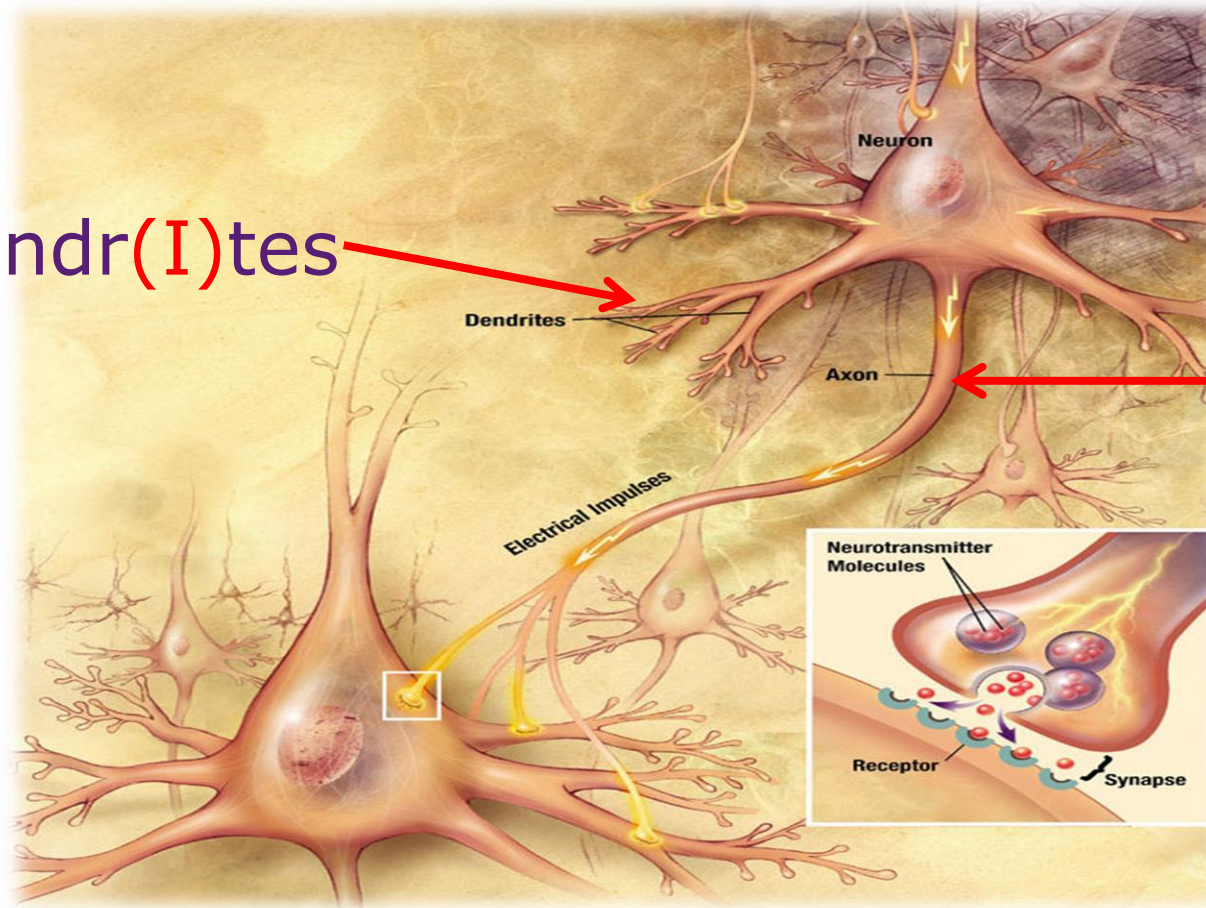
Deep neural networks learn hierarchical feature representations



Comment cela se passe dans le cerveau

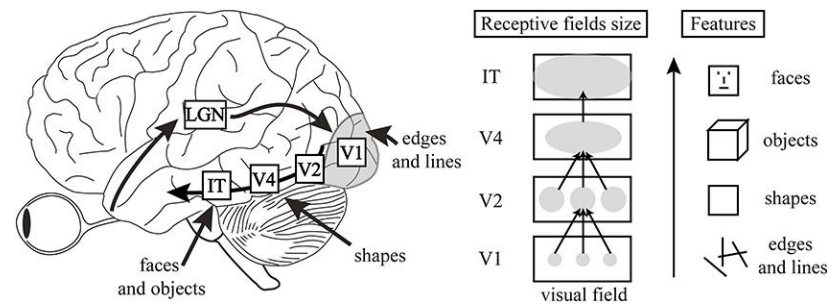
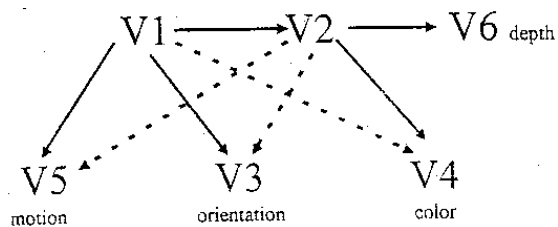
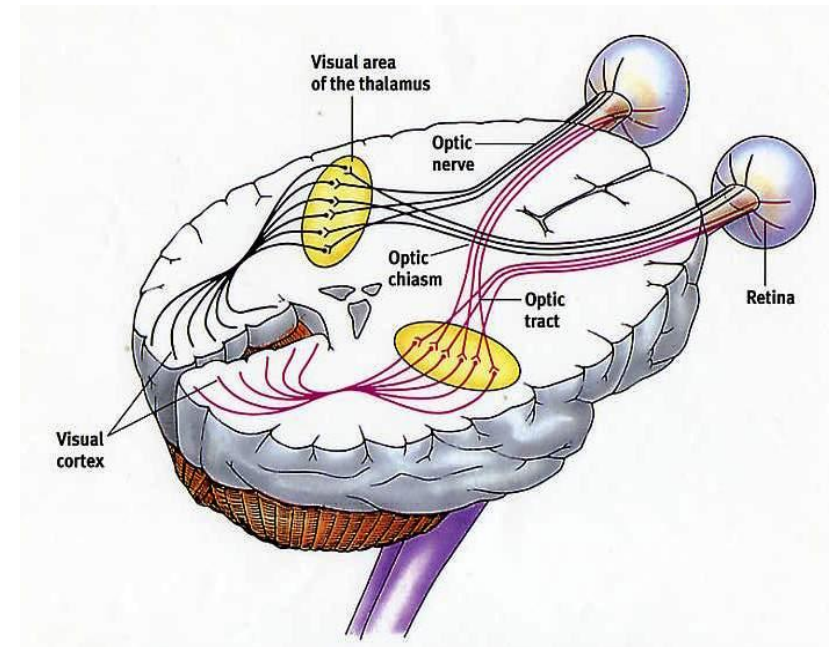
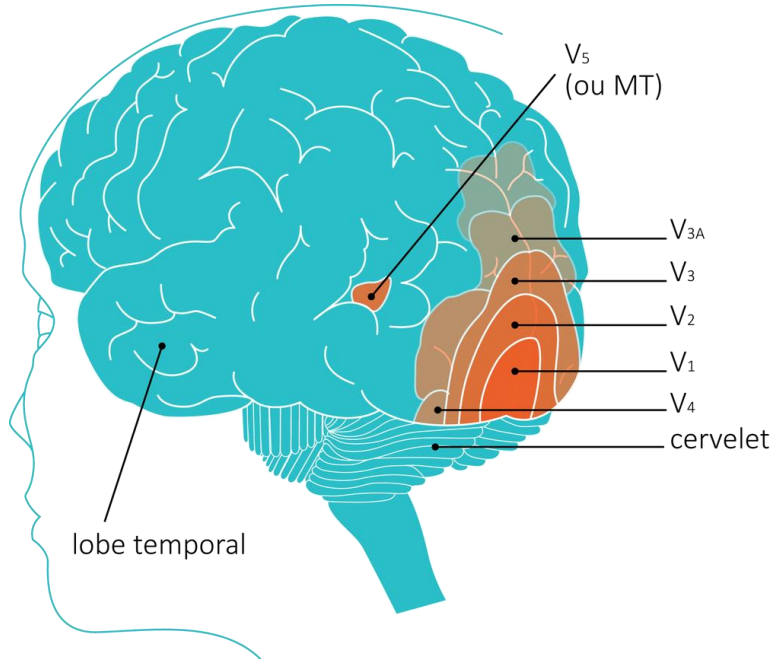


Dendr(I)tes

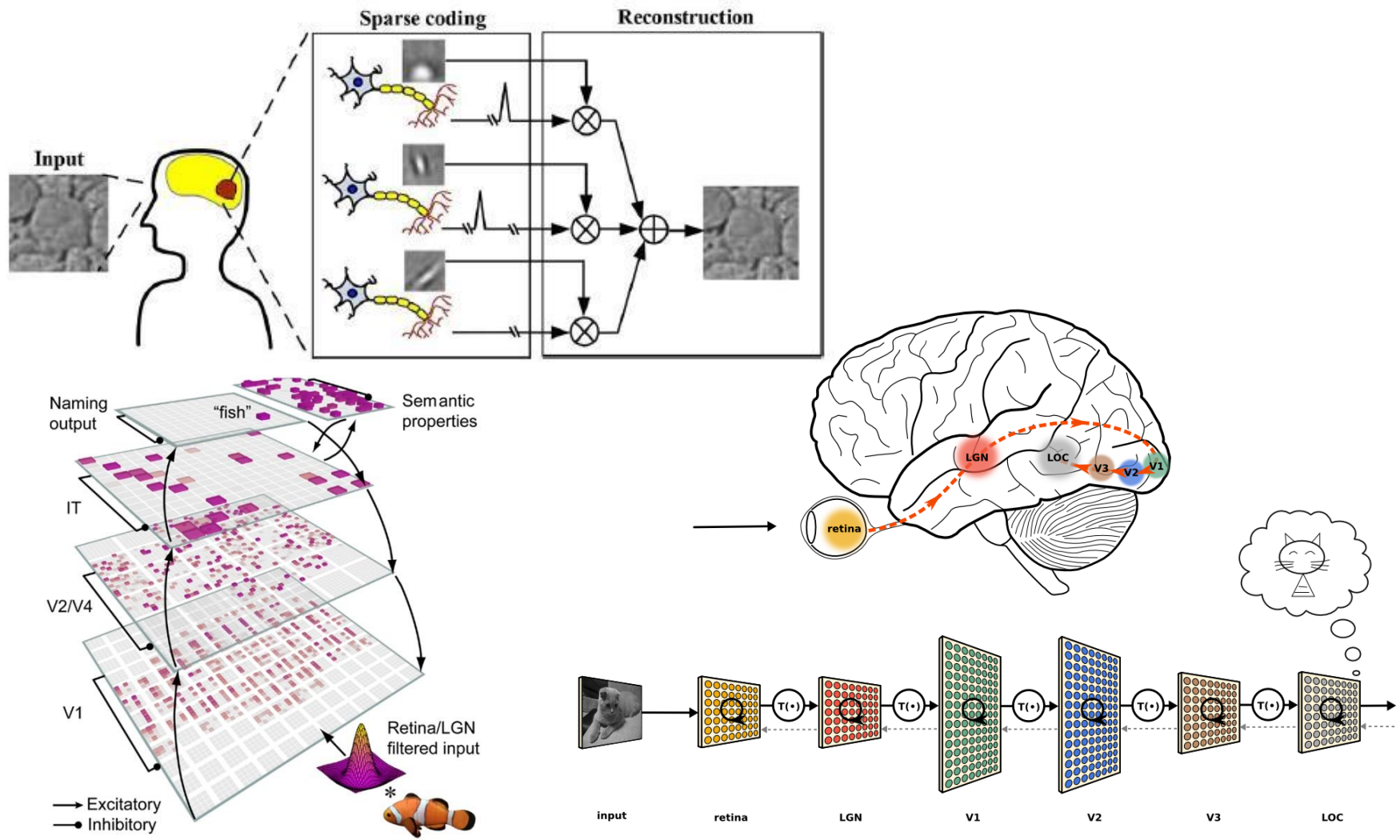


Ax(O)ne

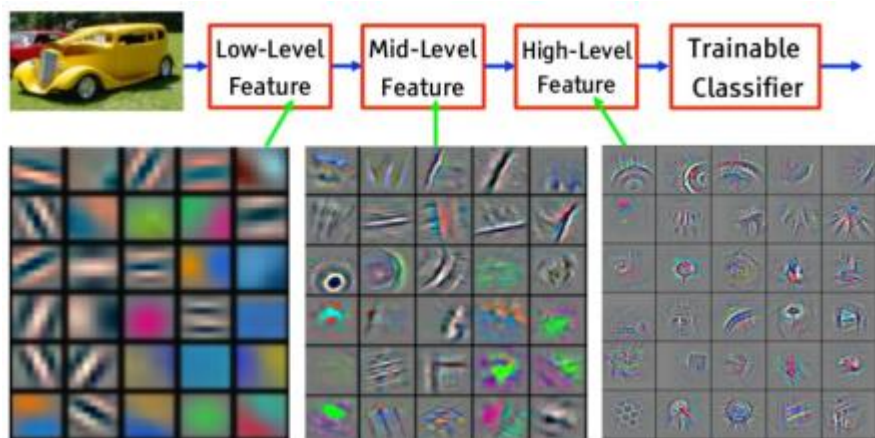
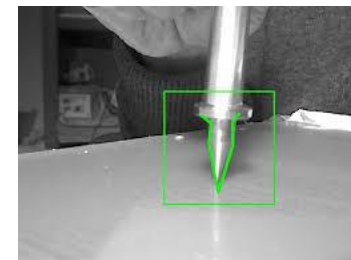
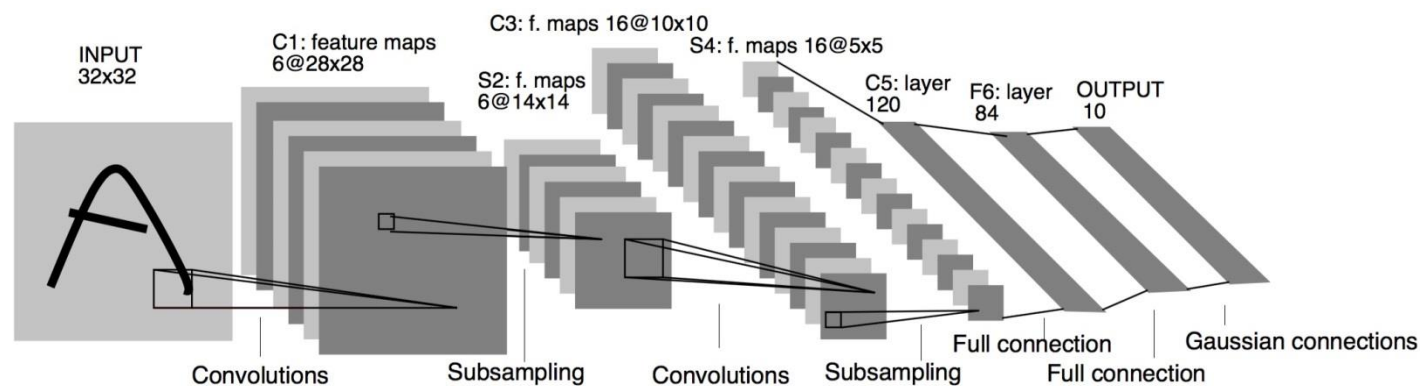
Traitement neuronal de la vision



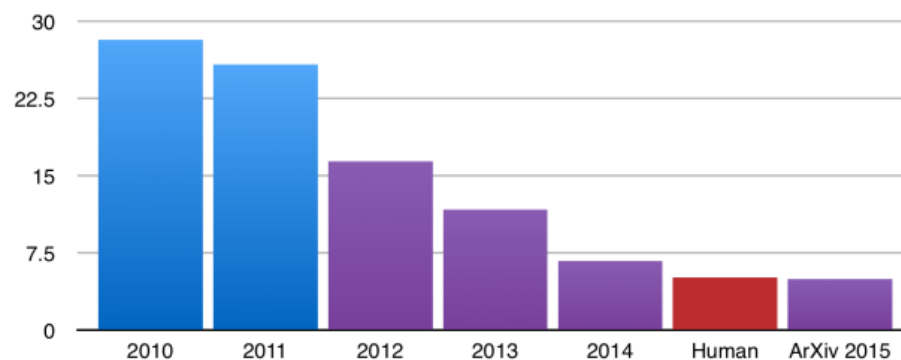
Fonctionnement de la vision



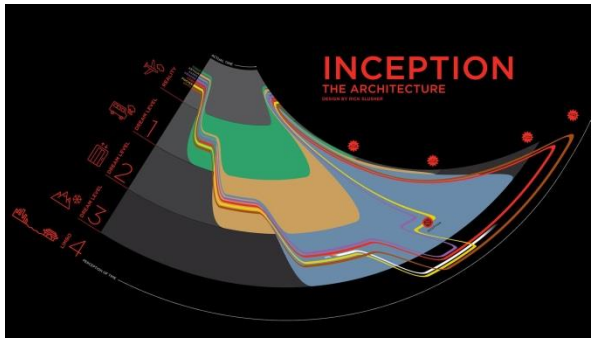
Lenet



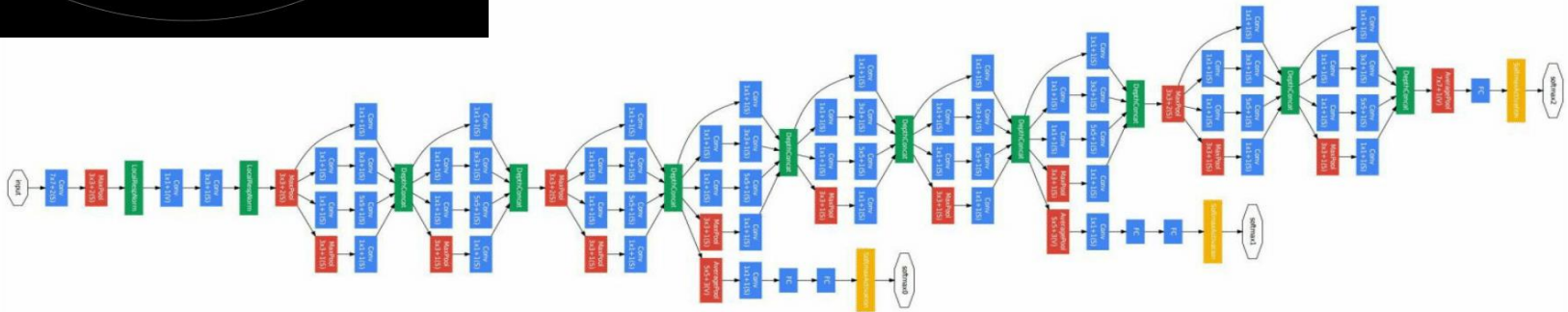
ILSVRC top-5 error on ImageNet



Inception (2015)



22 niveaux!



The Inception Architecture (GoogLeNet, 2015)

Lenet: Génération de représentant

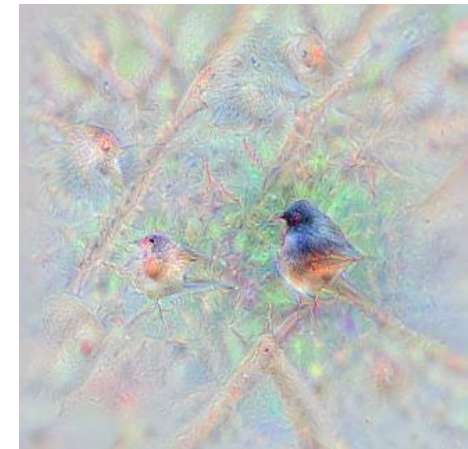
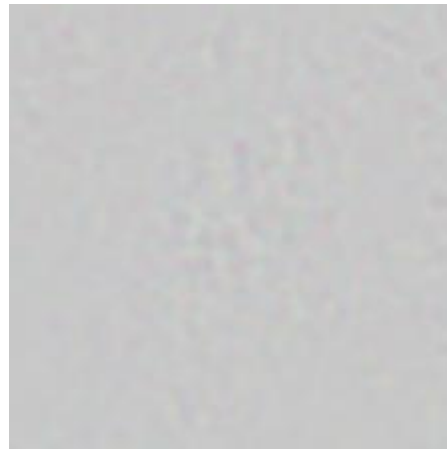


Going Deeper with Convolutions 2014

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich

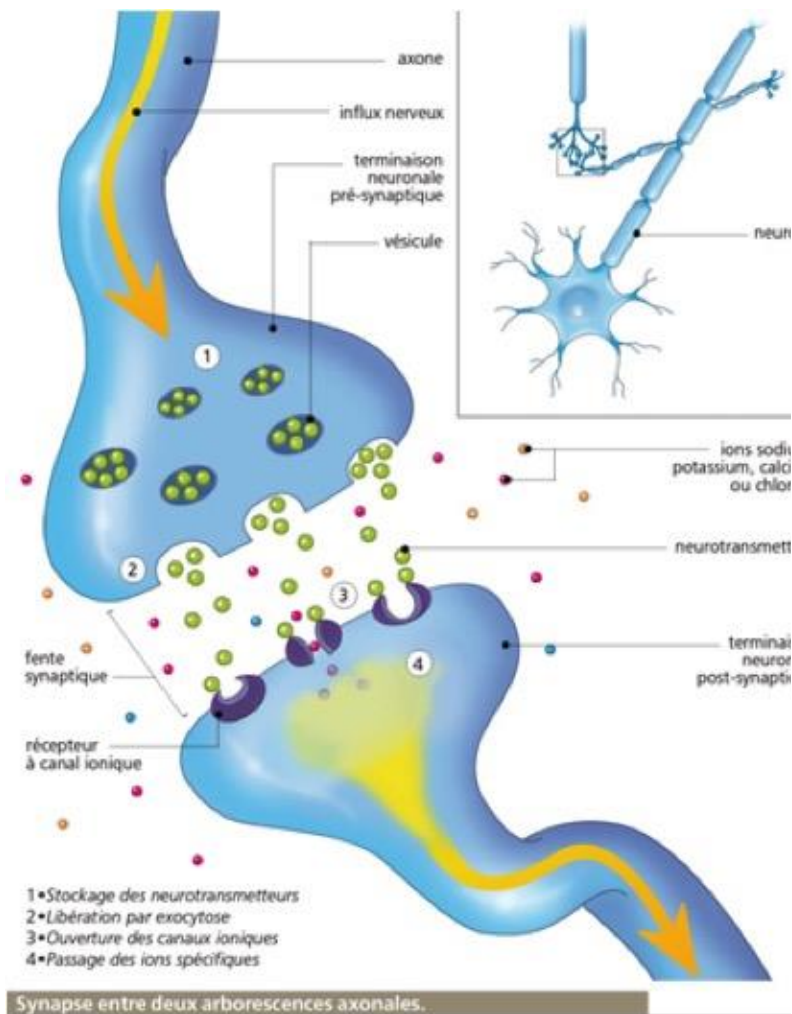
Understanding Neural Networks Through Deep Visualization 2015

Jason Yosinski, Jeff Clune, Anh Nguyen, Thomas Fuchs, Hod Lipson



GoogLeNet classes : <http://www.auduno.com/2015/07/29/visualizing-googlenet-classes/>

Une liaison synaptique en fonctionnement



Nature chimique	Nom du neurotransmetteur	Quelques sites de libération synaptique	Principaux rôles biologiques
Amines	Acétylcholine	Synapse neuromusculaire	Contraction des muscles striés Ralentissement cardiaque Motricité digestive Sécrétion
	Dopamine	Cerveau	Contrôle de la motricité (maladie de Parkinson) Contrôle de l'éveil comportemental
	Adrénaline	Bulbe rachidien	Augmentation rythme cardiaque, augmentation pression artérielle (stress, effort physique, danger...)
	Noradrénaline	Système orthosympathique Cortex cérébral Hypothalamus Bulbe rachidien	Stimulation cardiaque
	Sérotonine	Cortex cérébral Hypothalamus Cervelet Bulbe rachidien	Sommeil Thermorégulation Chronobiologie
Acides aminés	GABA (Acide Gamma-amino-butyrique)	Motoneurons de la moelle épinière Cerveau Moelle épinière	Neuromédiateur inhibiteur
	Glutamate et aspartate	Système nerveux central	Neuromédiateur exciteur
Peptides	Substance P	Cerveau Neuromédiateur des neurones sensitifs (peau vers moelle épinière)	Perceptions douloureuses : nociception
	Enképhalines	Cortex cérébral et moelle épinière	Analgésie : diminution de la sensibilité à la douleur

Chatbot contre chatboot



Robot parlant: SpeechToText + chatbot + TextToSpeech



Chatbot a la Watson



Reinforcement

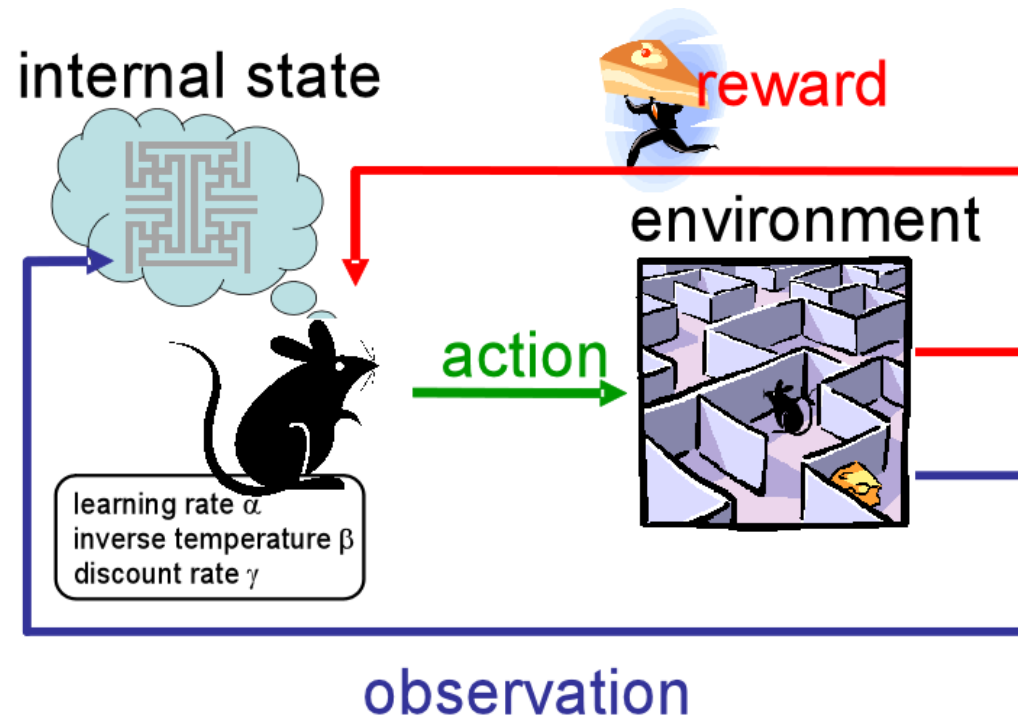


Copyright 2001 by Randy Glasbergen. www.glasbergen.com



“It’s a special hearing aid. It filters out criticism and amplifies compliments.”

Reinforcement learning



Les différents types de reinforcement



Approaches To Reinforcement Learning

Value-based RL

- ▶ Estimate the **optimal value function** $Q^*(s, a)$
- ▶ This is the maximum value achievable under any policy

Policy-based RL

- ▶ Search directly for the **optimal policy** π^*
- ▶ This is the policy achieving maximum future reward

Model-based RL

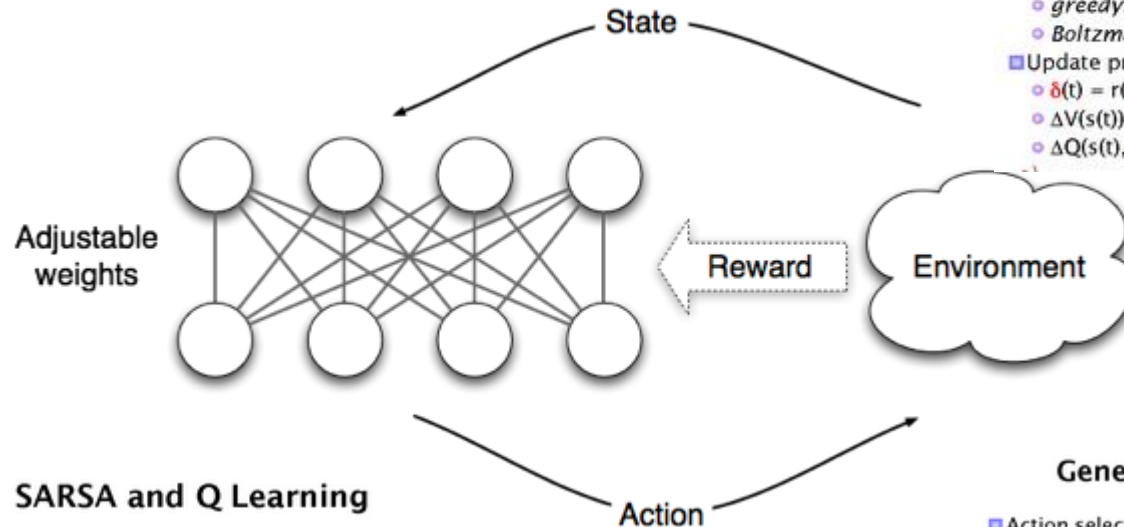
- ▶ Build a model of the environment
- ▶ Plan (e.g. by lookahead) using model

Reinforcement



Reinforcement Learning

- Predict reward: *value function*
 - $V(s) = E[r(t) + \gamma r(t+1) + \gamma^2 r(t+2) \dots | s(t)=s]$
 - $Q(s,a) = E[r(t) + \gamma r(t+1) + \gamma^2 r(t+2) \dots | s(t)=s, a(t)=a]$
- Select action *How to implement these steps?*
 - *greedy*: $a = \operatorname{argmax}_a Q(s,a)$
 - *Boltzmann*: $P(a|s) \propto \exp[\beta Q(s,a)]$
- Update prediction: *TD error*
 - $\delta(t) = r(t) + \gamma V(s(t+1)) - V(s(t))$
 - $\Delta V(s(t)) = \alpha \delta(t)$ *How to tune these parameters?*
 - $\Delta Q(s(t),a(t)) = \alpha \delta(t)$



SARSA and Q Learning

- Action value function
 - $Q(s,a) = E[r(t) + \gamma r(t+1) + \gamma^2 r(t+2) \dots | s(t)=s, a(t)=a]$
- Action selection
 - ϵ -greedy: $a = \operatorname{argmax}_a Q(s,a)$ with prob $1-\epsilon$
 - Boltzman: $P(a_i|s) = \exp[\beta Q(s,a_i)] / \sum_j \exp[\beta Q(s,a_j)]$
- SARSA: on-policy update
 - $\Delta Q(s(t),a(t)) = \alpha \{ r(t) + \gamma Q(s(t+1),a(t+1)) - Q(s(t),a(t)) \}$
- Q learning: off-policy update
 - $\Delta Q(s(t),a(t)) = \alpha \{ r(t) + \gamma \max_{a'} Q(s(t+1),a') - Q(s(t),a(t)) \}$

Generalized Q-learning Model

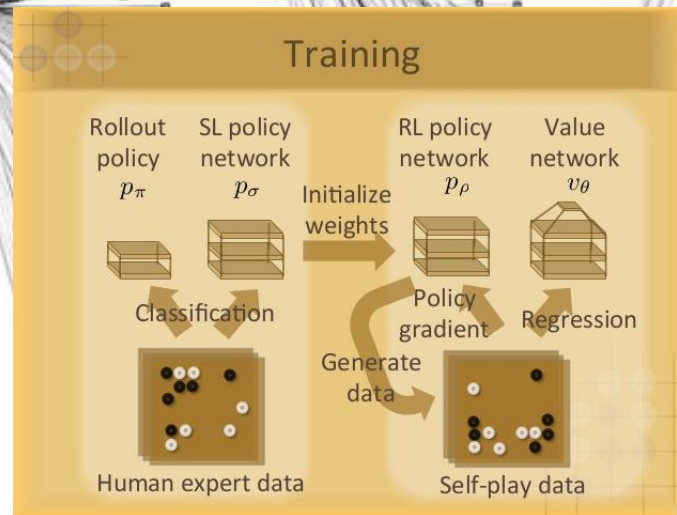
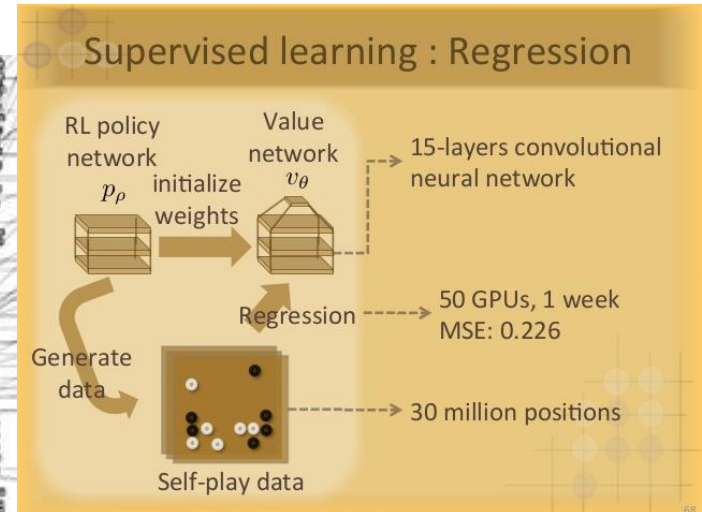
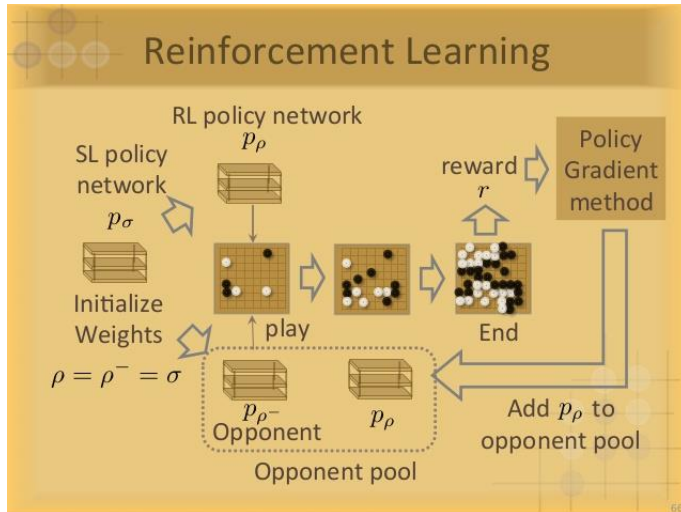
(Ito & Doya, 2009)

- Action selection

$$P(a(t)=L) = \exp Q_L(t) / (\exp Q_L(t) + \exp Q_R(t))$$
- Action value update: $i \in \{L, R\}$

$$Q_i(t+1) = \begin{cases} (1-\alpha_1)Q_i(t) + \alpha_1 \kappa_1 & \text{if } a(t)=i, r(t)=1 \\ (1-\alpha_1)Q_i(t) + \alpha_1 \kappa_2 & \text{if } a(t)=i, r(t)=0 \\ (1-\alpha_2)Q_i(t) & \text{if } a(t) \neq i, r(t)=1 \\ (1-\alpha_2)Q_i(t) & \text{if } a(t) \neq i, r(t)=0 \end{cases}$$
- Parameters
 - α_1 : learning rate
 - α_2 : forgetting rate
 - κ_1 : reward reinforcement
 - κ_2 : no-reward aversion

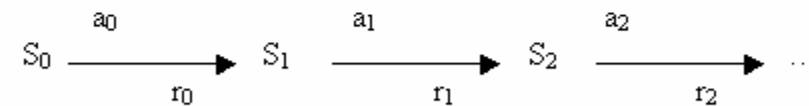
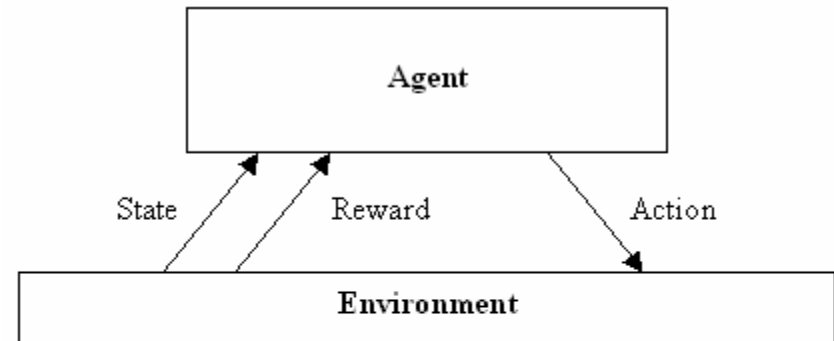
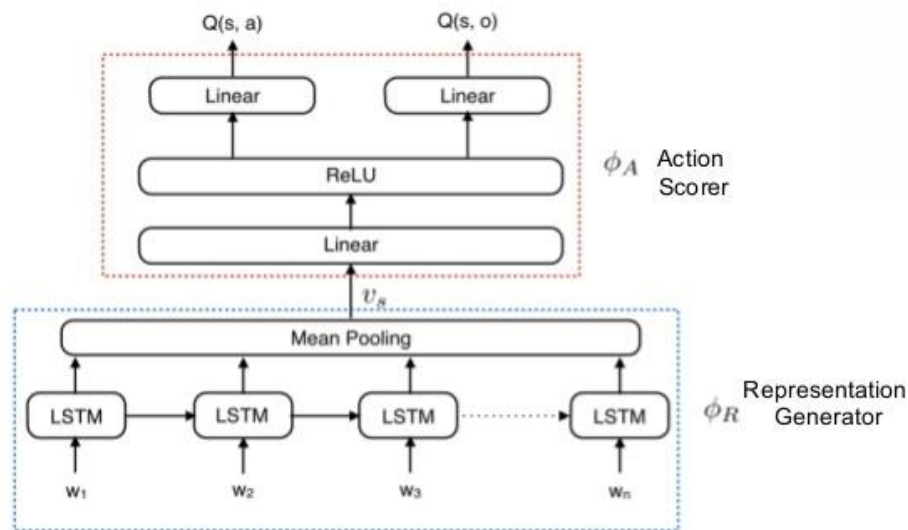
Reinforcement dans Alphago



Reinforcement profond



Deep Reinforcement Learning Framework



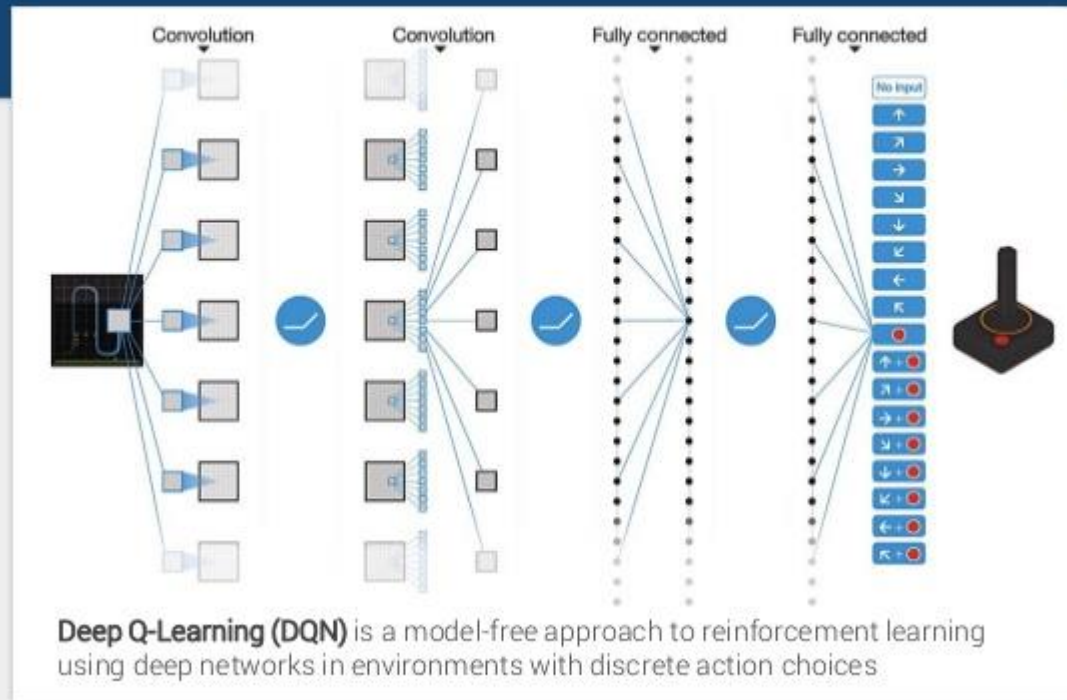
Goal: learn to choose actions that maximize:
 $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, where $0 \leq \gamma < 1$

Deep Q-Learning



Deep Learning in Google Research

DeepMind Deep Q-Learning





Toy video QA problem



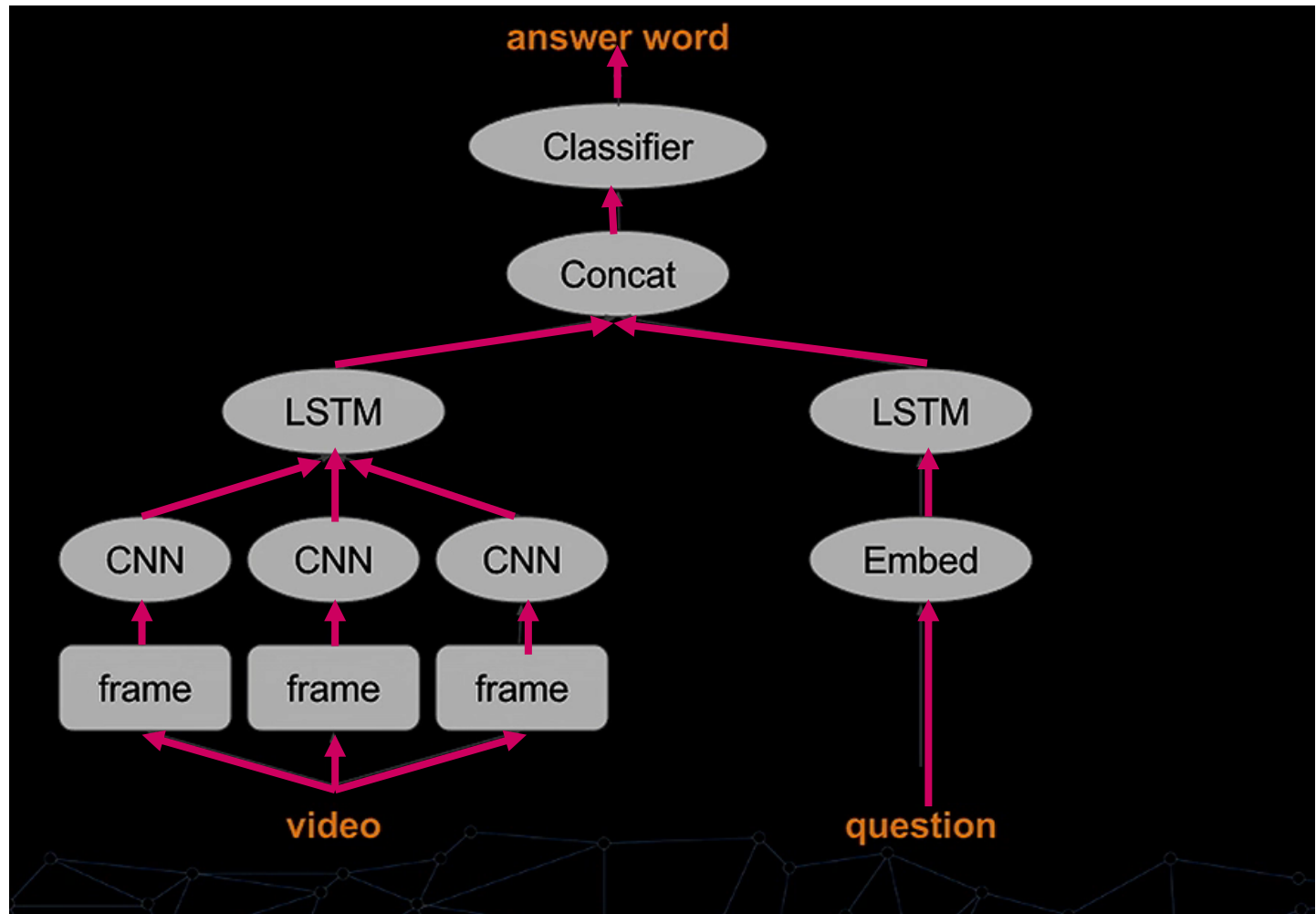
- > What is the woman doing?
- > **packing**
- > What is the color of her shirt?
- > **black**



La solution



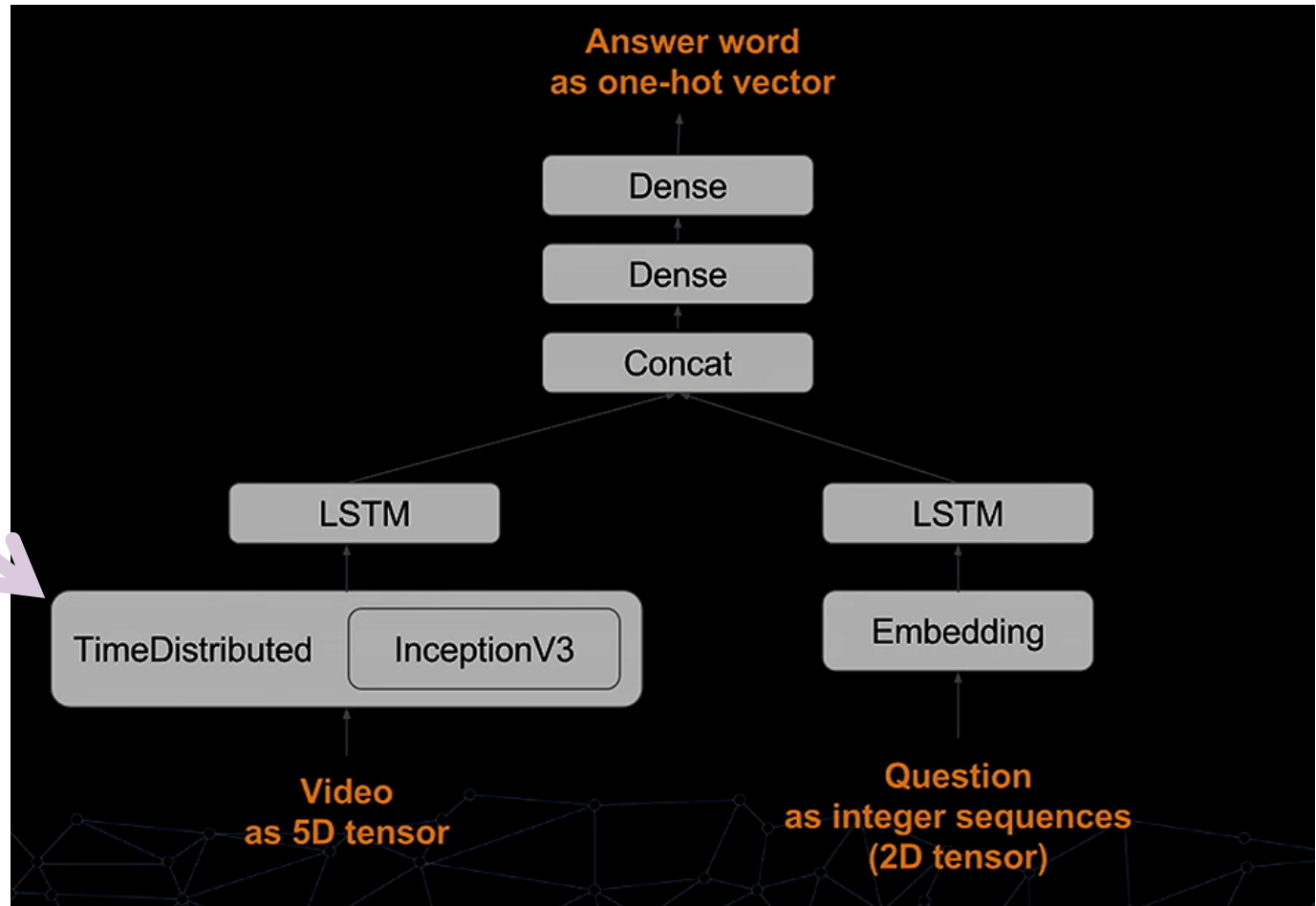
↑
Sémantique
Transportée
par un vecteur
↑



Traduite en Keras



Pre-trained



Le code du modèle (1 : partie video)



Pre-trained
Et
Non-trainable

```
video = tf.keras.layers.Input(shape=(None, 150, 150, 3))
cnn = tf.keras.applications.InceptionV3(weights='imagenet',
                                         include_top=False,
                                         pool='avg')
cnn.trainable = False
encoded_frames = tf.keras.layers.TimeDistributed(cnn)(video)
encoded_vid = tf.layers.LSTM(256)(encoded_frames)
```

Le modèle (2 : partie textuelle)



Transformation des mots en
vecteurs : vocabulaire = 10000

```
question = tf.keras.layers.Input(shape=(100), dtype='int32')  
x = tf.keras.layers.Embedding(10000, 256, mask_zero=True)(question)  
encoded_q = tf.keras.layers.LSTM(128)(x)
```

Remplir les 256 mots
Si nécessaire avec des blancs

Le modèle (3 : formation de la réponse)



```
x = tf.keras.layers.concat([encoded_vid, encoded_q])  
x = tf.keras.layers.Dense(128, activation=tf.nn.relu)(x)  
outputs = tf.keras.layers.Dense(1000)(x)
```

2 couches Linéaires
séparées
par une activation

Le modèle (conditions d'apprentissage)



Base de learning

```
model = tf.keras.models.Model([video, question], outputs)
model.compile(optimizer=tf.AdamOptimizer(),
              loss=tf.softmax_crossentropy_with_logits)
```

Méthode
d'optimisation

Spécification de la dernière couche d'activation

Le code en entier



```
video = tf.keras.layers.Input(shape=(None, 150, 150, 3))
cnn = tf.keras.applications.InceptionV3(weights='imagenet',
                                         include_top=False,
                                         pool='avg')

cnn.trainable = False
encoded_frames = tf.keras.layers.TimeDistributed(cnn)(video)
encoded_vid = tf.layers.LSTM(256)(encoded_frames)

question = tf.keras.layers.Input(shape=(100), dtype='int32')
x = tf.keras.layers.Embedding(10000, 256, mask_zero=True)(question)
encoded_q = tf.keras.layers.LSTM(128)(x)

x = tf.keras.layers.concat([encoded_vid, encoded_q])
x = tf.keras.layers.Dense(128, activation=tf.nn.relu)(x)
outputs = tf.keras.layers.Dense(1000)(x)

model = tf.keras.models.Model([video, question], outputs)
model.compile(optimizer=tf.AdamOptimizer(),
              loss=tf.softmax_crossentropy_with_logits)
```

Intégration de TensorFlow et de Keras



```
import tensorflow as tf
from keras import backend as K
from keras.layers import Dense
from keras.objectives import categorical_crossentropy

# this placeholder will contain our input digits, as flat vectors
img = tf.placeholder(tf.float32, shape=(None, 784))
# Keras layers can be called on TensorFlow tensors:
# fully-connected layer with 128 units and ReLU activation
x = Dense(128, activation='relu')(img)
x = Dense(128, activation='relu')(x)
```

Keras TensorFlow



KETE Combo rocks

Img : defini dans tensorflow

Comment on lance un learning



On change de méthode d'optimisation (si on veut)

```
# for a multi-class classification problem
model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(X_train, Y_train, nb_epoch=5, batch_size=32)
```

C'est parti !

Utiliser un modèle calibré



Evaluate your performance in one line:

```
loss_and_metrics = model.evaluate(X_test, Y_test, batch_size=32)
```

Or generate predictions on new data:

```
classes = model.predict_classes(X_test, batch_size=32)  
proba = model.predict_proba(X_test, batch_size=32)
```

Utiliser des modèles pre-trained



Extract features with VGG16

```
from keras.applications.vgg16 import VGG16
from keras.preprocessing import image
from keras.applications.vgg16 import preprocess_input
import numpy as np

model = VGG16(weights='imagenet', include_top=False)

img_path = 'elephant.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)

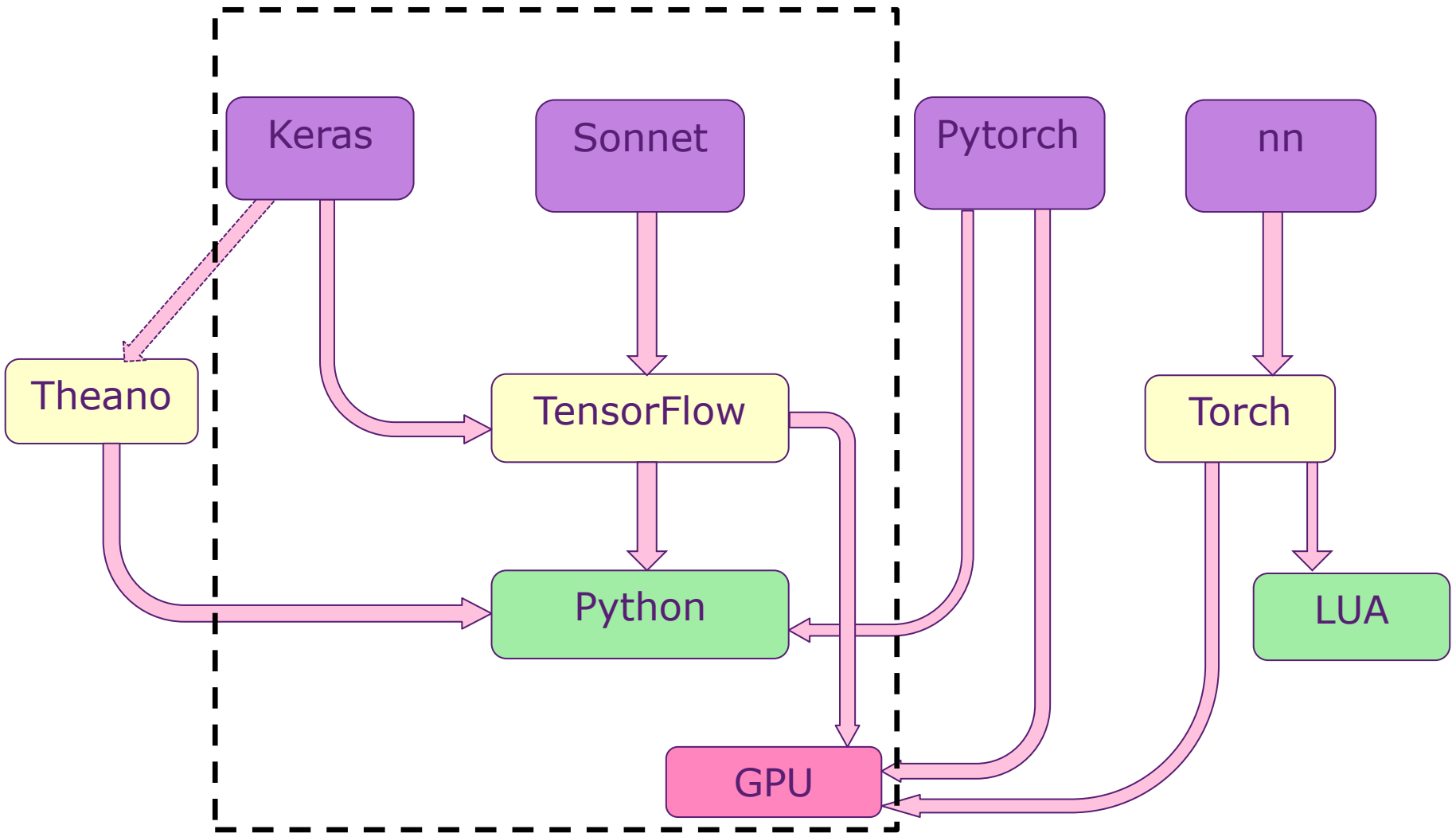
features = model.predict(x)
```

On charge
un modèle

On extrait
Les
features

On ne charge
que le ConvNet

Environnement de dev



Keras (approche fonctionnelle)



```
from keras.layers import Input, Dense
from keras.models import Model

# This returns a tensor
inputs = Input(shape=(784,))

# a layer instance is callable on a tensor, and returns a tensor
x = Dense(64, activation='relu')(inputs)
x = Dense(64, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)

# This creates a model that includes
# the Input layer and three Dense layers
model = Model(inputs=inputs, outputs=predictions)
model.compile(optimizer='rmsprop',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(data, labels) # starts training
```

Activation déclarée dans la couche

S'appuie sur

- Ou bien Tensorflow
- Ou bien Theano

Sonnet (Deepmind 2017)



Approche fonctionnelle

```
def custom_build(inputs, is_training, keep_prob):  
    """A custom build method to wrap into a sonnet Module."""  
    outputs = snt.Conv2D(output_channels=32, kernel_shape=4, stride=2)(inputs)  
    outputs = snt.BatchNorm()(outputs, is_training=is_training)  
    outputs = tf.nn.relu(outputs)  
    outputs = snt.Conv2D(output_channels=64, kernel_shape=4, stride=2)(outputs)  
    outputs = snt.BatchNorm()(outputs, is_training=is_training)  
    outputs = tf.nn.relu(outputs)  
    outputs = snt.BatchFlatten()(outputs)  
    outputs = tf.nn.dropout(outputs, keep_prob=keep_prob)  
    outputs = snt.Linear(output_size=10)(outputs)  
    return outputs
```

Package nn de Torch

Learning to Learn in TensorFlow

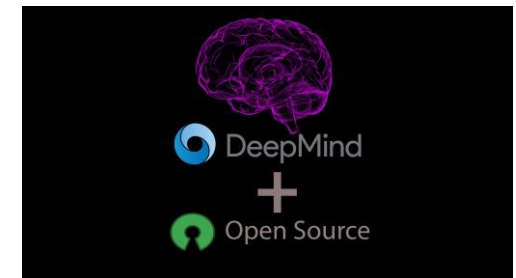
Dependencies

- TensorFlow >=1.0
- Sonnet >=1.0

```
class MetaOptimizer(object):
```

```
    """Learning to learn (meta) optimizer.
```

```
    Optimizer which has an internal RNN which takes as input, at each iteration,  
    the gradient of the function being minimized and returns a step direction.  
    This optimizer can then itself be optimized to learn optimization on a set of  
    tasks.
```





Deux premières idées d'applications

Akinator Financier:

- Sélectionner un produit financier sur la base d'un questionnaire dynamique, avec enrichissement automatique de la base de connaissance

Pricer ultra rapide a base de Réseau neuronal :

- Pricer en situation (sur une tablette en face du client)
- Comparer un ensemble de structures



« A peine née , aussitôt morte »

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)
        self.conv2_drop = nn.Dropout2d()
        self.fc1 = nn.Linear(320, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x):
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
        x = x.view(-1, 320)
        x = F.relu(self.fc1(x))
        x = F.dropout(x, training=self.training)
        x = self.fc2(x)
        return F.log_softmax(x)
```

Package nn de Torch

Malgré :

- Python
- Arbre syntaxique dynamique
- Torchvision
- GPU + rapide

Mais Standard = Tensorflow ..

Generated C#



L'ancêtre fondateur

LUA

```
--[[Model]]--  
  
model = nn.Sequential()  
model:add(nn.Convert(ds:ioShapes(), 'bf')) -- to batchSize x nFeature (also type converts)  
  
-- hidden layers  
inputSize = ds:featureSize()  
for i,hiddenSize in ipairs(opt.hiddenSize) do  
  model:add(nn.Linear(inputSize, hiddenSize)) -- parameters  
  if opt.batchNorm then  
    model:add(nn.BatchNormization(hiddenSize))  
  end  
  model:add(nn.Tanh())  
  if opt.dropout then  
    model:add(nn.Dropout())  
  end  
  inputSize = hiddenSize  
end  
  
-- output layer  
model:add(nn.Linear(inputSize, #(ds:classes())))  
model:add(nn.LogSoftMax())
```

Accès vers c efficace

```
static int l_sin (lua_State *L) {  
  double d = lua_tonumber(L, 1); /* get argument */  
  lua_pushnumber(L, sin(d)); /* push result */  
  return 1; /* number of results */  
}
```